NS Client FIPS Mode Support

References
1. Introduction and Background
    1.1 Introduction to FIPS
    1.2 Background
    1.3 Project Scope
2. FIPS Module and OpenSSL 3.x
3. Design Considerations
4. Design Details
    4.1 Configurations
        4.1.1 config FipsMode via Registry (Windows) and Preferences (MacOS)
        4.1.2 config FipsMode via Command Line
    4.2 OpenSSL Version and FIPS Module Version Selections
    4.3 FIPS Mode Enabling
        FIPS Mode Enabling Workflow
        4.3.1 FIPS Library Initialization
        4.3.2 OpenSSL Provider Registration
        4.3.3 Self-Test and Integrity Verification
        4.3.4 Operational Mode Activation
        4.3.5 Error Handling
        4.3.6 FIPS Mode Enabling in NS Agent and NS Service
    4.4 Cipher Suites Enforcement for FIPS Mode
        4.4.1 Cipher Suites
        4.4.2 TLS Handshaking Flow and Cipher Suite Negotiation
        4.4.3 TLS 1.2 and TLS 1.3
            4.4.3.1 Cipher Suite Selection for TLS 1.2 (FIPS 140-3 Compliance)
            4.4.3.2 Cipher Suite Selection for TLS 1.3 (FIPS 140-3 Compliance)
            4.4.3.3 FIPS 140-3 Cipher Suite Compliance Summary
        4.4.4 NS Client Modules Using TLS Connection
    4.5 Conditional Runtime Tests
        4.5.1 What Conditional Runtime Tests Are
        4.5.2 Types of Conditional Runtime Tests
        4.5.3 Conditional Runtime Tests Error Handling
    4.6 UI Mockups
    4.7 Logging
    4.8 Installation and Uninstallation
        4.8.1 Installation
        4.8.2 Uninstallation
Appendix 1: OpenSSL 3.x + FIPS Support Matrix (as of August 5, 2025)
Appendix 2: OpenSSL 3.0.8 provider algorithm support matrix
    1. Symmetric Ciphers
    2. Hash Functions (Digests)
    3. Public Key Algorithms (Asymmetric)
    4. Message Authentication Codes (MACs)
    5. Key Derivation Functions (KDFs)
Appendix 3: Key Differences Between FIPS-140-2 and FIPS-140-3

References

- NPLAN-4241: Support FIPS-140-3 compliance requirements IN PROGRESS
- NPLAN-4147: FIPS 140-3 support for NPA Client BACKLOG
- **NIST CMVP** – Cryptographic Module Validation Program: https://csrc.nist.gov/projects/cryptographic-module-validation-program
- **NIST FIPS 140-3 Standard** – Security Requirements for Cryptographic Modules: https://doi.org/10.6028/NIST.FIPS.140-3
- **OpenSSL FIPS 140-3 User Guide** (OpenSSL.org) https://www.openssl.org/docs/man3.0/man7/fips_module.html
- **OpenSSL Provider Documentation** (OpenSSL.org) https://www.openssl.org/docs/man3.0/man7/provider.html
- **OpenSSL EVP API Documentation** https://www.openssl.org/docs/man3.0/man3/EVP_DigestInit_ex.html

1. Introduction and Background

1.1 Introduction to FIPS

The Federal Information Processing Standards (FIPS) are U.S. government requirements for cryptographic modules used to protect sensitive information. FIPS 140-3 , the current standard, defines the security requirements for cryptographic modules, including algorithm validation, operational modes, and secure key management. Many government agencies and regulated industries mandate that software and hardware products operate in a FIPS-validated mode to ensure compliance with these standards. This document outlines the design specification for enabling and managing FIPS Mode support within Netskope Client (NSC) product. In FIPS Mode, the product will enforce the exclusive use of validated cryptographic modules and algorithms, ensuring that all encryption, hashing, and key management operations meet the prescribed security requirements without compromising performance, stability, or usability.

1.2 Background

From a market perspective, increasing regulatory scrutiny, compliance mandates, and heightened customer security expectations are accelerating the demand for FIPS-compliant solutions across both public and private sectors. Organizations in government, finance, healthcare, and defense now routinely include FIPS support as a mandatory procurement criterion. Many competitors in the endpoint security space have already introduced FIPS

Mode capabilities, positioning them to capture these compliance-driven opportunities. Delivering this capability will position our product as a leading choice, address the requirements of regulated customers, and strengthen our standing in security-sensitive markets.

1.3 Project Scope

This project covers the implementation of FIPS Mode support for the Netskope Client endpoint on Windows and macOS platforms. The iOS and Android platforms are excluded from this phase, as OpenSSL 3.x does not currently provide a FIPS 140-3 validated cryptographic module for these operating systems. Support for Linux will be addressed in a subsequent Phase 2 initiative.

The Netskope Private Access (NPA) component, being an integral part of the NS Client, is also included within the scope of this project to ensure consistent FIPS compliance across all relevant client modules.

Modules such as BWAN and epDLP are outside the scope of this project and will not be included in the current implementation phase.

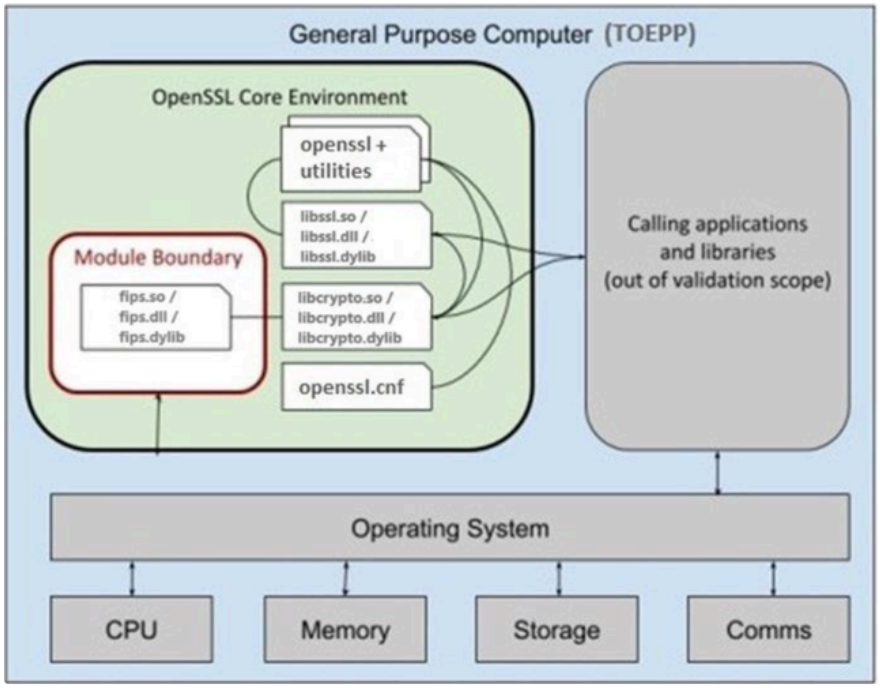
2. FIPS Module and OpenSSL 3.x

OpenSSL 3.x introduces a provider-based architecture that separates cryptographic implementations into modular components, including a dedicated FIPS provider (Appendix 2 shows the comparison among providers). The FIPS provider contains only FIPS 140-validated algorithms and operates under strict compliance controls defined by the standard. When the FIPS provider is loaded and activated, all cryptographic operations performed by the application must use algorithms from this validated set to maintain compliance. This architecture allows the endpoint security product to dynamically switch between default and FIPS-validated cryptographic providers, enabling both flexibility for general deployments and strict adherence to regulatory requirements when operating in FIPS Mode. Leveraging OpenSSL 3.x ensures long-term support, access to updated validated modules, and alignment with industry best practices for secure cryptographic operations.

The FIPS provider in OpenSSL 3.x is delivered as a distinct module, sometimes referred to as the “FIPS Object Module,” which has undergone validation through the U.S. National Institute of Standards and Technology (NIST) Cryptographic Module Validation Program (CMVP). Each validated module is associated with a specific certificate number, which serves as formal evidence of compliance. Validation applies to specific operating systems, CPU architectures, and build configurations; using the module outside of these parameters would

fall outside the scope of the certification. For this reason, NS Client app must ensure that the deployed cryptographic binaries exactly match the validated module's build parameters. Additionally, the integration must include mechanisms to verify at runtime that the FIPS provider is loaded, active, and in an approved operational state, ensuring both technical compliance and audit readiness.

The figure below is from the FIPS official documentation and it illustrates the concept of module boundary and interaction with NS Client app.



OpenSSL FIPS Provider Block Diagram

3. Design Considerations

To maintain strict compliance with FIPS 140-3 requirements, NSC must incorporate several key design elements related to the FIPS provider's operation.

First, the product should implement mandatory runtime self-tests on the FIPS module each time it is initialized to verify its integrity and proper functionality, as required by the standard. Any failure in these tests must immediately disable FIPS Mode and trigger appropriate alerts or error handling to prevent insecure cryptographic operations.

Second, the system must enforce strict algorithm usage policies, ensuring that only FIPS-validated algorithms provided by the FIPS module are utilized when FIPS Mode is enabled. Attempts to use non-validated algorithms should be blocked and logged for audit purposes.

Comprehensive logging of FIPS Mode activation, cryptographic operations, and any error conditions must be maintained to support security audits and regulatory reporting.

Finally, the design should include seamless fallback and notification mechanisms that enable administrators to quickly identify compliance issues and remediate them without disrupting normal product functionality when FIPS Mode is disabled. Together, these design considerations ensure the endpoint security product meets the operational, security, and compliance expectations associated with FIPS Mode support.

4. Design Details

4.1 Configurations

We will introduce an integer-type registry key **FipsMode** to allow administrators to configure this feature. Using a registry key provides greater flexibility compared to a single tenant-level flag. This approach enables fine-grained policy control, allowing administrators to enable or enforce FIPS mode selectively based on criteria such as user group, organizational unit, or device attributes. For example, with a MDM server, administrators can configure different policies depending on the operating system version, device type, or compliance requirements of a particular department. This registry-based configuration ensures that organizations can align FIPS enforcement with their security and operational needs while maintaining scalability across diverse environments.

Non-FIPS Mode (FipsMode=0)

All cryptographic operations are performed without enforcing FIPS 140-validated algorithms or modules. This mode offers maximum flexibility but does not guarantee compliance.

FIPS Mode (or, Strict FIPS Mode, FipsMode=1)

Only FIPS 140-validated algorithms and modules are permitted. NS Client app will enter into fail-close mode if FIPS mode initialization failed , ensuring full compliance.

Permissive FIPS Mode (FipsMode=2)

FIPS 140-validated algorithms and modules are used by default when available. If FIPS mode initialization fails, NS Client app falls back to non-FIPS Mode, allowing compatibility while preferring compliance.

Mode	Security	Compliance	Compatibility
Non-FIPS Mode	Uses all available algorithms,	No FIPS 140 compliance guaranteed	Maximum compatibility with legacy

	including non-validated ones		systems and libraries
Strict FIPS Mode FipsMode=1	Restricts use to only FIPS 140-validated algorithms and modules	Full FIPS 140 compliance; non-FIPS algorithms rejected	Limited compatibility; applications requiring non-FIPS algorithms may fail
Permissive FIPS Mode FipsMode=2	Prefers FIPS 140-validated algorithms but allows fallback to non-FIPS when necessary	Partial compliance — FIPS algorithms used when possible, but fallback weakens guarantees	Balanced compatibility; applications continue working even if FIPS algorithms are unavailable

4.1.1 config FipsMode via Registry (Windows) and Preferences (MacOS)

Windows	HKEY_LOCAL_MACHINE/SOFTWARE/Netskope/PersistentSettings/FipsMode (DWORD)
MacOS	/Library/Preferences/com.netskope.provisioning.plist/FipsMode (Integer)

4.1.2 config FipsMode via Command Line

Windows	msiexec /I STAgent.msi /l*v "C:\users\bhu\stagent-install.log" tenant=qa domain=de.goskope.com **fipsmode=1** installmode=IDP
MacOS	sudo -S ./nsclientconfig.sh 0 0 0 idp de.goskope.com qa 1 mode="scheme" **FipsMode=2** preferephemeral="true" httpmethod=get

4.2 OpenSSL Version and FIPS Module Version Selections

FIPS compliance in OpenSSL 3.x is determined by the provider module that's loaded at runtime—not by the core `libcrypto` / `libssl` libraries the application links against. In practice, that means you must ship the exact validated FIPS provider binary and its `fipsmodule.cnf`, load the OpenSSL config before enabling FIPS, and operate on platforms/architectures listed in the provider's Security Policy. This separation lets you update the core for bug fixes while keeping the validated crypto boundary in the provider.

OpenSSL 3.x currently has two validated FIPS provider lines: the OpenSSL 3.0.8/3.0.9 provider (certificate #4811, FIPS 140-2) and the OpenSSL 3.1.2 provider (certificate #4985, FIPS 140-3).

Because FIPS 140-2 is being phased out with a validation sunset on September 21, 2026, and many regulated buyers already specify FIPS 140-3, we will standardize on the FIPS 140-3–validated OpenSSL FIPS provider 3.1.2 to avoid recertification churn and align with current compliance expectations.

For Windows, MacOS and Linux, for the core libraries (`libcrypto` / `libssl`), we will use OpenSSL 3.5.2 to benefit from the latest security patches, while loading the validated 3.1.2 FIPS provider at runtime. We'll verify provider compatibility in staging, ship the exact `fips.dll/.so` referenced by our `openssl.cnf`, generate `fipsmodule.cnf` via `fipsinstall`, and confirm that our target OS/architecture is covered by the provider's Security Policy.

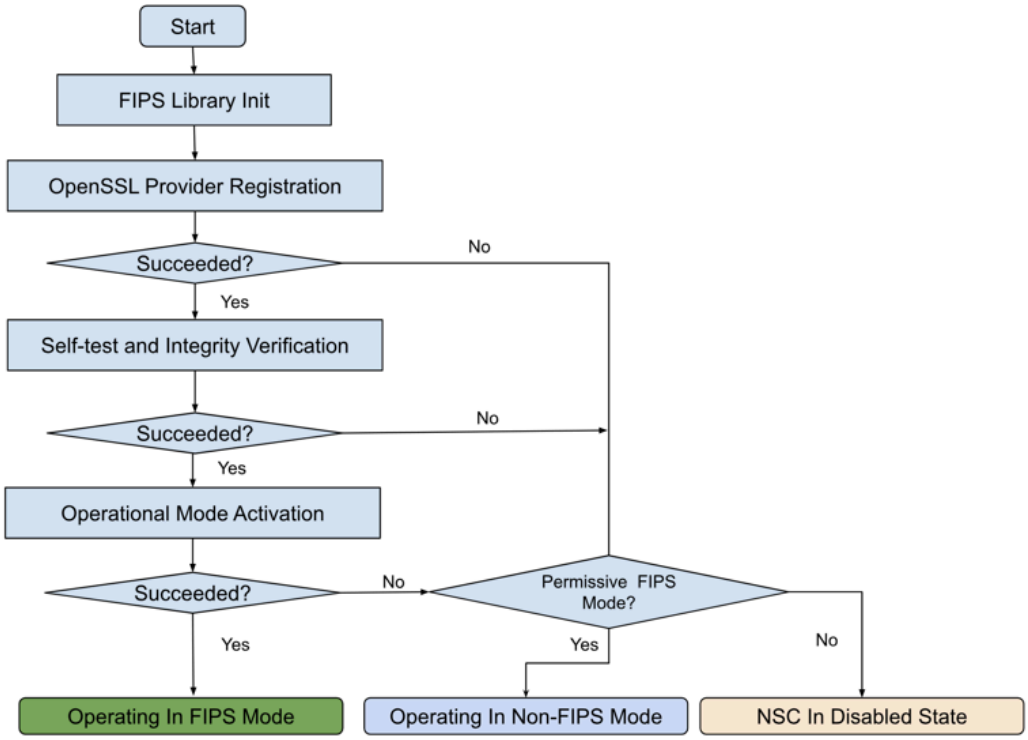
For Android and iOS, we'll use the OpenSSL 3.5.2 FIPS provider to maintain API uniformity and pick up the latest fixes, but we will not claim FIPS certification on these platforms. The core reason is that the OpenSSL 3.1.2 FIPS-140-3 validated provider does not cover Android or iOS in its Operational Environments, so shipping it there wouldn't make those builds certified anyway. Using the 3.5.2 provider delivers current security patches—while we can clearly document Android/iOS as non-certified and avoid any compliance assertions until a validated provider for mobile OS exists.

The Table below summarizes the OpenSSL version and FIPS module version selections:

OS	OpenSSL Version	FIPS Module Version	FIPS Certified	NSC Releases?
Windows (64bit)	3.5.2	3.1.2 (FIPS 140-3)	✅	

MacOS	3.5.2	3.1.2 (FIPS 140-3)	✅	
Linux	3.5.2	3.1.2 (FIPS 140-3)	✅	
Android	3.5.2	3.5.2	❌	
iOS	3.5.2	3.5.2	❌	

4.3 FIPS Mode Enabling



FIPS Mode Enabling Workflow

4.3.1 FIPS Library Initialization

Upon NS Client startup, the designated FIPS cryptographic library (`.so` on Linux/macOS or `.dll` on Windows) is loaded into process memory. This library contains the FIPS 140-validated cryptographic module that enforces approved algorithms and operational boundaries as defined by NIST.

4.3.2 OpenSSL Provider Registration

During the OpenSSL initialization phase, the client explicitly loads the FIPS Provider module using the OpenSSL 3.x provider interface. The provider is registered within the OpenSSL

context, ensuring that all cryptographic operations—including key generation, hashing, and encryption—are routed through FIPS-approved implementations.

4.3.3 Self-Test and Integrity Verification

Once the FIPS Provider is loaded, the module automatically executes a series of power-on self-tests (POSTs), including integrity checks (HMAC or SHA256 digest verification of the module binary) and known-answer tests (KATs) for critical algorithms. These tests validate that the module has not been tampered with and that its cryptographic functions produce expected results.

4.3.4 Operational Mode Activation

If all self-tests pass successfully, the module transitions into Approved (FIPS) mode, and a status flag is set within the OpenSSL context to indicate compliance. Subsequent cryptographic operations are executed exclusively within the validated boundary.

4.3.5 Error Handling

If provider registration, self-tests, integrity checks, or operational mode initialization fails, the NS Client will respond according to the FIPS mode configured by the administrator:

- **Strict FIPS Mode**:
  The NS Client enters a disabled state, blocking all non-FIPS cryptographic operations. This fail-close behavior ensures that the application cannot continue operating in a non-compliant state, thereby maintaining strict adherence to FIPS 140 requirements.
- **Permissive FIPS Mode:**
  The NS Client automatically falls back to Non-FIPS Mode, continuing normal operation with standard cryptographic algorithms. This approach prioritizes compatibility while still preferring FIPS compliance when available.

4.3.6 FIPS Mode Enabling in NS Agent and NS Service

The NS Agent and NS Service run as two separate processes, each dynamically linked to its own instance of the OpenSSL library. As a result, FIPS Mode enabling must be performed independently within both components to ensure consistent cryptographic enforcement across the entire NS Client architecture.

If either process fails to enable FIPS mode—due to provider registration errors, integrity check failures, or module initialization issues—the overall FIPS state of the NS Client will depend on the configured operating mode:

- Under Permissive FIPS Mode, the client will gracefully fall back to Non-FIPS Mode, maintaining functionality while logging the compliance downgrade.
- Under Strict FIPS Mode, the client will transition into a disabled state, preventing further cryptographic operations to preserve compliance integrity. This dual-process validation ensures that both the Agent and Service operate under the same verified FIPS boundary and eliminates the risk of mixed or partial compliance.

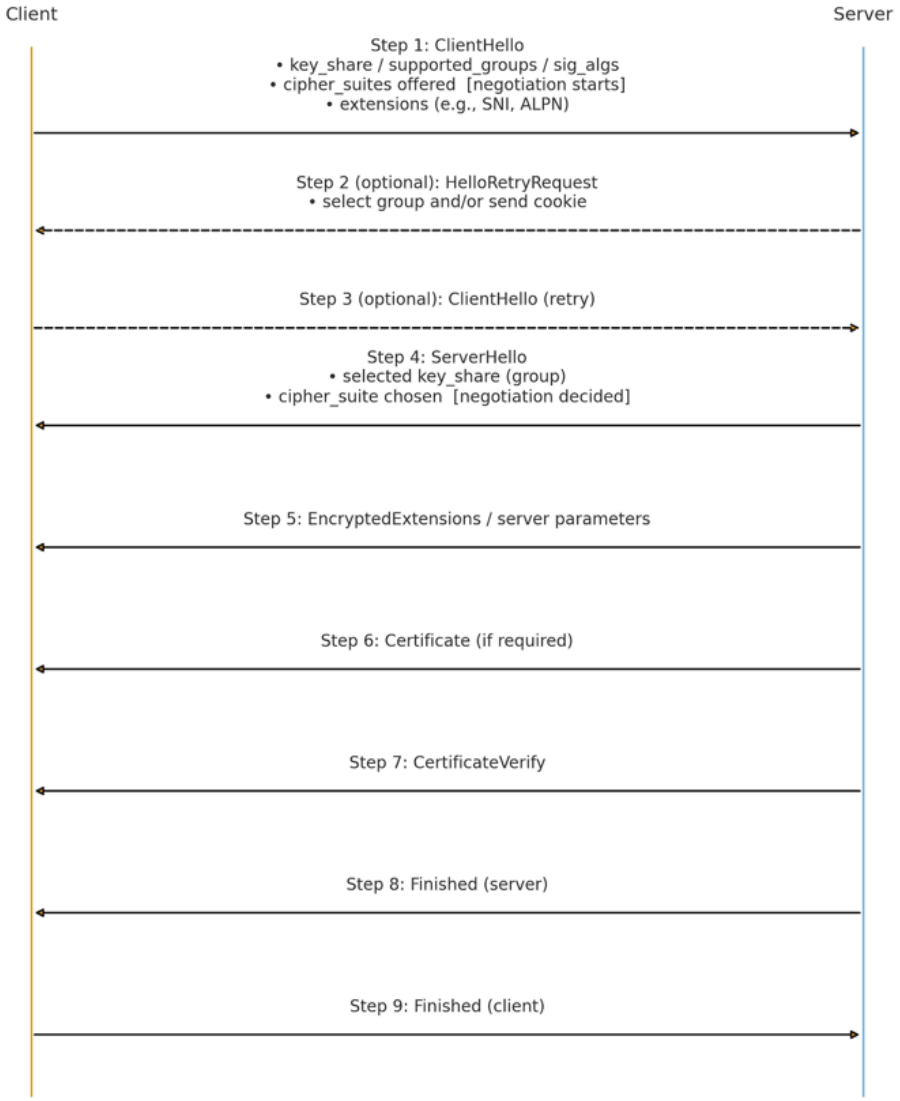
4.4 Cipher Suites Enforcement for FIPS Mode

4.4.1 Cipher Suites

A cipher suite in Transport Layer Security (TLS) defines the specific set of algorithms used to secure communications between a client and a server. Each cipher suite specifies algorithms for key exchange, authentication, encryption, and message authentication (integrity). During the TLS handshake, the client and server negotiate and agree on the strongest mutually supported cipher suite to ensure both security and compatibility.

The chosen cipher suite ultimately determines the cryptographic strength, performance, and compliance level of the secure connection.

4.4.2 TLS Handshaking Flow and Cipher Suite Negotiation



Cipher suites negotiation occurs between Step 1 (offered in ClientHello) and Step 4(decided in ServerHello). Dashed steps (2-3) show the optional HelloRetryRequest (HRR) path.

For FIPS 140-3 operation, NS Client must both:

(1) run in a FIPS-validated crypto module and

(2) restrict suites to those built from FIPS-approved algorithms.

4.4.3 TLS 1.2 and TLS 1.3

Most modules within the NS Client are compatible with both TLS 1.2 and TLS 1.3. Each TLS version defines its own set of FIPS 140-3–approved cipher suites, reflecting differences in cryptographic design and compliance requirements between the two versions. During the TLS

handshake, the NS Client and server negotiate the highest mutually supported TLS version and select the strongest FIPS-approved cipher suite available.

4.4.3.1 Cipher Suite Selection for TLS 1.2 (FIPS 140-3 Compliance)

When operating in FIPS 140-3 compliant mode, TLS 1.2 must be configured to use only cipher suites that rely exclusively on FIPS-approved algorithms. These typically include AES-GCM or AES-CBC for encryption, SHA-256 or stronger for message authentication, and RSA or ECDSA for authentication and key exchange (with ECDHE preferred to ensure forward secrecy). Examples of compliant cipher suites include TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 and TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256.

Cipher suites using deprecated or non-approved algorithms—such as RC4, MD5, DES, 3DES, or anonymous key exchange (NULL authentication)—must be disabled. Proper configuration ensures that all cryptographic operations are executed by FIPS-validated modules, maintaining end-to-end assurance of compliance.

4.4.3.2 Cipher Suite Selection for TLS 1.3 (FIPS 140-3 Compliance)

TLS 1.3 simplifies the cipher suite structure, enhancing security and compliance. It removes separate key exchange and authentication algorithms from the suite, handling them via ephemeral elliptic-curve Diffie–Hellman (ECDHE) key exchange and certificate-based authentication within the protocol. Only a few modern, FIPS-approved cipher suites remain valid: TLS_AES_128_GCM_SHA256, TLS_AES_256_GCM_SHA384, and TLS_CHACHA20_POLY1305_SHA256 (the last only if supported by a FIPS-validated implementation).

4.4.3.3 FIPS 140-3 Cipher Suite Compliance Summary

TLS Version	FIPS-Compliant Cipher Suites	Non-Compliant Cipher Suites (Examples)	Notes

TLS 1.2	`TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`
	`TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA`
		`TLS_RSA_WITH_RC4_128_SHA`
		`TLS_RSA_WITH_3DES_EDE_CBC_SHA`
		`TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` (non-AEAD)
			FIPS requires AES (128/256-bit), SHA-2 (≥ SHA-256), and ECDHE/RSA or ECDSA for key exchange. Legacy algorithms like RC4, DES,

	`256` `TLS_DHE_RSA_WITH_AES_256_GCM_SHA384`		MD5, and non-AEAD modes are disallowed.

TLS 1.3	`TLS_AES_128_GCM_SHA256`
	`TLS_AES_256_GCM_SHA384`
	`TLS_CHACHA20_POLY1305_SHA256`*
		None defined at protocol level (TLS 1.3 deprecates all legacy ciphers)	TLS 1.3 inherently enforces strong FIPS-approved primitives. *CHACHA20_POLY1305* is allowed only if implemented in a validated FIPS module. OpenSSL 3.1.2 excludes this cipher suite so we will not use it in NS Client.

4.4.4 NS Client Modules Using TLS Connection

NS Client Module	Module Functionality	Note
tlspipe	NPA tunnel connection to gateway	Currently support TLS 1.2. only.
nsHttpClient	nsDownloader, connect to addonman, secure configure, longpoll connection, etc.	Both TLS 1.2 and TLS 1.33
nssl	SWG tunnel connection to gateway	Both TLS 1.2 and TLS 1.33
AppProbStatsComm	P-DEM app probing connection	Currently support TLS 1.2. only

For all five modules, the allowed cipher suites will be configured dynamically based on the selected FIPS operation mode. Specifically:

- **Non-FIPS Mode**:
  The system will continue using the existing cipher suite configuration without any restrictions or modifications.
- **Strict FIPS Mode**:
  Only FIPS 140-3–compliant cipher suites will be enabled, ensuring that all cryptographic

operations strictly adhere to validated algorithms and compliance requirements.

- **Permissive FIPS Mode**:
  If FIPS mode initialization succeeds, the configuration will mirror that of Strict FIPS Mode. If initialization fails, the system will automatically revert to the Non-FIPS Mode cipher suite configuration, maintaining compatibility while preferring compliance when possible.

4.5 Conditional Runtime Tests

4.5.1 What Conditional Runtime Tests Are

Unlike Power-On Self-Tests (POSTs), which run only once when the FIPS module is loaded, Conditional Runtime Tests are automatically executed whenever certain cryptographic events occur during runtime— to continuously ensure the module's ongoing integrity.

These tests are embedded inside the OpenSSL FIPS provider logic and cannot be disabled when the provider is in FIPS Mode.

4.5.2 Types of Conditional Runtime Tests

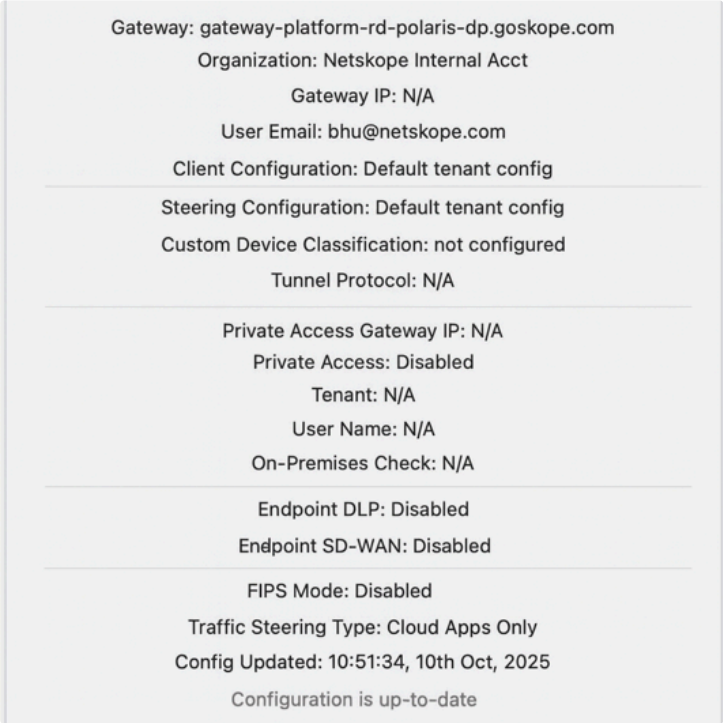
Event Trigger	Test Type	Description	Typical Trigger in NS Client
Key Pair Generation	*Pairwise Consistency Test*	After generating an RSA, ECDSA, DH, or ECDH key pair, the module performs a round-trip test (e.g., sign → verify or encrypt → decrypt) using internal test data. If the operation fails, the key is discarded, and the module enters an error state.	When NS Client provisions new certificates or regenerates ephemeral keys.
Random Number Generation (RNG)	*Continuous RNG Test*	Every new random number output is compared to the previous output; if they match, it indicates a failure in the DRBG.	When generating nonces, session keys, or initialization vectors.

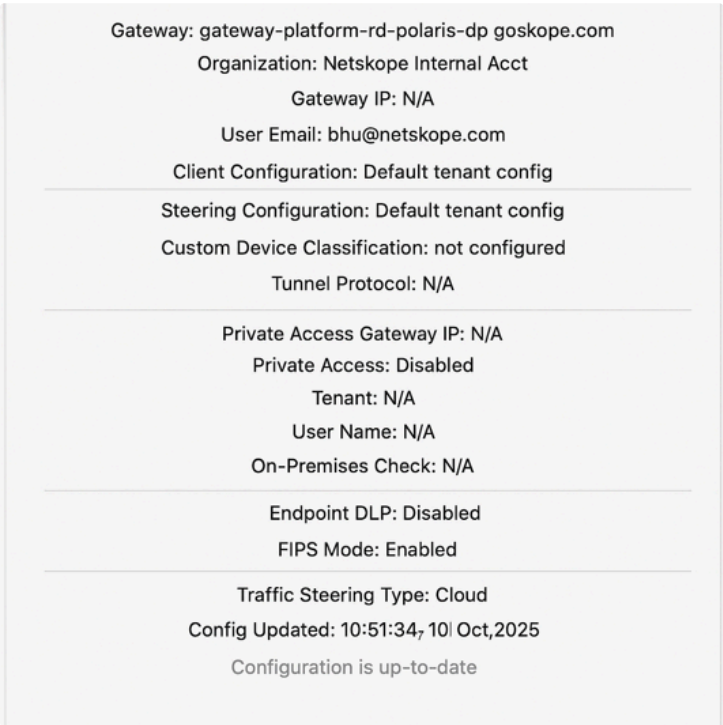
If any of these conditional tests fail, OpenSSL internally sets the provider into an "Error State." Any subsequent `EVP_*` or `RAND_*` operation will return failure (`0`) immediately.

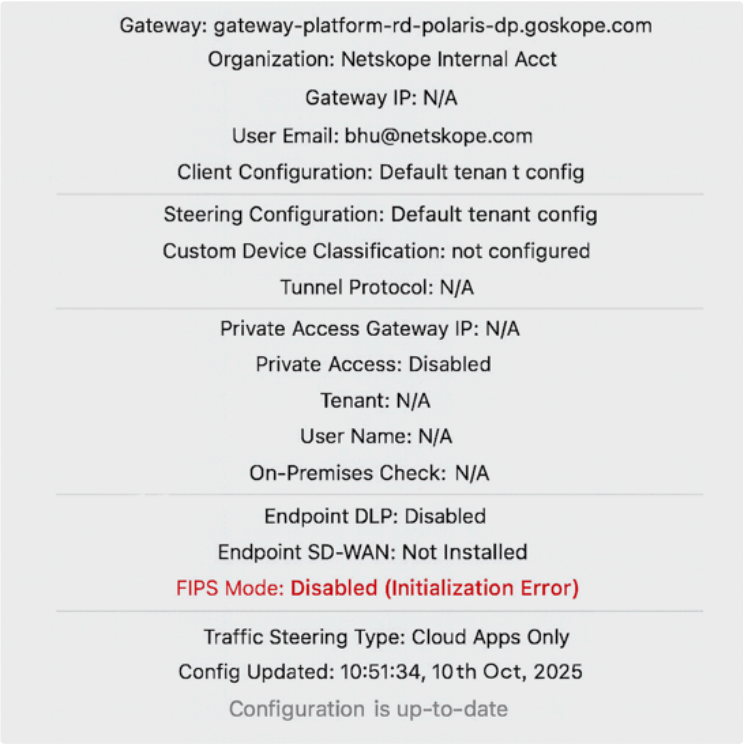
4.5.3 Conditional Runtime Tests Error Handling

When the provider in an “Error State”, any subsequent EVP_* or RAND_* operation will return failure (0) immediately. The error info will be saved to NS Client debug logs files.

4.6 UI Mockups







4.7 Logging

To facilitate troubleshooting and verification, all FIPS mode–related events will be logged in `nsdebuglog.log` and `nsAppUI.log` files, each entry prefixed with the tag **"FIPS:"** for easy identification. For example:

2025/10/10 18:23:07.941576 stAgentNE p53747 t11527 info peer.cpp:194 FIPS: Enabled successfully.

4.8 Installation and Uninstallation

4.8.1 Installation

During NS Client installation or upgrade, the FIPS provider dynamic library (`fips.dll` on Windows or `fips.dylib` on macOS) and the accompanying FIPS module configuration file (`fipsmodule.cnf`) shall be installed in a subdirectory named `openssl` within the application's installation directory.

4.8.2 Uninstallation

When the NS Client application is uninstalled, the `openssl` subdirectory shall be removed, and all FIPS Mode–related registry entries shall be cleaned up upon completion of the uninstallation

process.

Appendix 1: OpenSSL 3.x + FIPS Support Matrix (as of August 5, 2025)

OpenSSL Version	Platform / OS	Architecture	FIPS-validated Provider?	Certificate #	`fips.so` / `fips.dll` Buildable?
3.0.8 – 3.0.9	Windows	x86_64	✅ Yes (FIPS 140-2)	4811	✅ Yes (if built with validated sources)
3.0.8 – 3.0.9	Windows	x86	❌ Not FIPS-certified	n/a	⚠️ Not provided by default
3.0.8 – 3.0.9	macOS	x86_64 / arm64	✅ Yes (FIPS 140-2)	4811	✅ Yes (if built correctly)
3.0.8 – 3.0.9	Linux	x86_64	✅ Yes (FIPS 140-2)	4811	✅ Yes
3.1.0 – 3.1.1	Windows / macOS / Linux	varied	⚠️ Supported modules only	4811	⚠️ Works if backward-compatible
3.1.2	Linux (only)	x86_64	✅ Yes (FIPS 140-3)	new cert	✅ Yes
3.1.2	Windows / macOS	x86_64 / arm64	❌ Not FIPS-certified	—	⚠️ Not provided by default
3.1.2	MacOS (x86_64)	x86_64	✅ Yes (FIPS 140-3 backward capability)	new certs	✅ Yes
3.1.2	Windows (x86_64)	x86_64	✅ Yes (FIPS 140-3 backward capability)	new certs	✅ Yes
3.1.2	Linux (x86_64)	x86_64	✅ Yes (FIPS 140-3	new certs	✅ Yes

			backward capability)		
3.2.x / later	Windows / macOS / Mobile	varied	❌ Not validated	—	⚠️ Only dev builds if manually done

Appendix 2: OpenSSL 3.0.8 provider algorithm support matrix

1. Symmetric Ciphers

Algorithm	Legacy	Default	FIPS
AES-128/192/256-CBC	✅	✅	✅
AES-CTR	✅	✅	✅
AES-GCM	✅	✅	✅
AES-CCM	✅	✅	✅
AES-XTS		✅	✅
AES-WRAP (RFC3394)		✅	✅
3DES (Triple DES)	✅	✅	✅*
DES / DES-EDE	✅		
ChaCha20		✅	
RC2	✅		
RC4	✅		
IDEA	✅		
CAST	✅		

2. Hash Functions (Digests)

Algorithm	Legacy	Default	FIPS
SHA1	✅	✅	✅
SHA2-224/256/384/512	✅	✅	✅
SHA3-224/256/384/512		✅	✅
SHAKE128/256		✅	
MD5	✅	✅	
MD2	✅		
MD4	✅		
RIPEMD160	✅	✅	
SM3		✅	
Whirlpool		✅	

3. Public Key Algorithms (Asymmetric)

Algorithm	Legacy	Default	FIPS
RSA (PKCS#1, OAEP, PSS)	✅	✅	✅
DSA	✅	✅	✅*
DH (Diffie-Hellman)	✅	✅	✅
ECDSA (P-256/P-384/P-521)		✅	✅
Ed25519 / Ed448		✅	
X25519 / X448		✅	

SM2		✅	
GOST R 34.10-2001	✅		

4. Message Authentication Codes (MACs)

Algorithm	Legacy	Default	FIPS
HMAC (SHA-1/2/3)	✅	✅	✅
CMAC (AES, 3DES)	✅	✅	✅
Poly1305		✅	
GMAC (AES)		✅	✅
SipHash		✅	

5. Key Derivation Functions (KDFs)

Algorithm	Legacy	Default	FIPS
PBKDF2		✅	✅
HKDF		✅	✅
TLS1-PRF		✅	✅
X963KDF		✅	✅
SSKDF		✅	✅
KBKDF		✅	✅
SSHKDF		✅	
TLS1.3 KDF		✅	✅

Appendix 3: Key Differences Between FIPS-140-2 and FIPS-140-3

Aspect	**FIPS-140-2**	**FIPS-140-3**

Standard Basis	Based on **ISO/IEC 19790:2006**	Based on **ISO/IEC 19790:2012** and **ISO/IEC 24759:2017**
Effective Dates	Mandatory for new submissions until Sept 21 2020; sunset period until Sept 21 2026 (for existing certs)	Became mandatory for new submissions on Sept 22 2020
Security Levels	Levels 1–4, defined mainly for physical and logical security	Same four levels, but with refined and updated requirements aligned with ISO standards
Testing Documentation	NIST-specific Derived Test Requirements (DTRs)	Uses ISO/IEC 24759 test requirements, harmonized with global standards
Physical Security Requirements	Less explicit about environmental failure protection; physical inspection guidelines were NIST-specific	More detailed environmental failure protection and tamper response requirements per ISO; updates to role separation, authentication, and side-channel guidance
Algorithm Requirements	Lists algorithms in Annexes; periodically updated by NIST SP 800-131A transitions	Algorithm requirements are now in separate ISO and NIST publications, making updates more flexible
Self-tests	Power-up and conditional tests required; specific NIST-defined patterns	Still requires self-tests, but aligns methods and terminology with ISO/IEC; increased

		emphasis on integrity and conditional tests
International Recognition	Recognized under the older **CSTL/CMVP** agreements, mostly North America	Harmonized with **ISO/IEC** enables broader international recognition and potential for cross-jurisdiction reuse
Operational Environment (OE) Definition	Vendor Affirmation allowed more flexibility for adding new OEs without retesting	OE coverage is more formal; additions typically require lab testing unless explicitly permitted in IGs
Policy & Guidance	NIST-specific Implementation Guidance for interpretation	Still has CMVP Implementation Guidance, but designed to be consistent with ISO interpretations